

Choosing the Right OS Structure and Understanding Its System Calls

**Submitted by:
Karl Zyrus M. Pantaleon**

**Polytechnic University of the Philippines Sto. Tomas Campus
A Case Problem Analysis Paper Presented to the Instructor of
Operating Systems**

**Submitted To:
Dr. Melani L. Castillo**

September 14, 2026

Scenario:

You are part of the Systems Development Team in a mid-sized software company tasked to design a new specialized operating system for a client. The client runs a cloud-based e-commerce platform that requires:

- High reliability and security (to protect customer data)

- Efficient resource management (to handle thousands of transactions per second)
- Flexibility for future upgrades and modular enhancements
- Ability to interact smoothly with applications (e.g., database, file handling, user accounts)

I. Identify and Compare

1.MODULAR

Modular because one of the main advantage of using modular structure is that it is flexible for future upgrades. New modules can be loaded or removed while the system is running and it will not cause a disrupt to the system. And you don't need to reboot the whole OS just to add something new, that's why it makes it easier to add features or support new hardware as the e-commerce grows. This is backed up by research showing that modular and microservices-based architectures actually improve e-commerce scalability and make it faster to integrate with systems like payment gateways and CRM tools, compared to traditional monolithic designs (Guntakandla, 2025). Each module in modular structure talks to the core kernel so the different parts of the platform can work together without needing to know how each other works internally.

The disadvantage of it is that this structure has an security risk, since modules can be loaded while the system is running, a bad or malicious module could destroy or harm our whole system. Second is debugging is much harder since the module get loaded and unloaded different times, it makes it harder to know where does the error comes from. Lastly, compatibility issues can happen a module might not work well with a newer kernel version or with other modules, especially if updates aren't properly synced (Jafari, 2025).

2.MICROKERNEL

Microkernel keeps only the most basic functions inside the kernel, like memory management, process scheduling, and communication between programs. Everything else, like file systems and device drivers, runs separately as its own process . This separation is the main idea behind microkernels, and it makes the system more secure and reliable, because a problem in one part does not automatically break the whole system.

This idea fits a cloud-based e-commerce platform, which needs to be reliable and secure. The same idea is used in real e-commerce systems today services like login, payment, and notifications are kept separate, so if one of them fails , it does not affect something important like payment processing (IJEETR, 2022.). Because the kernel itself is small and simple, there is less chance for attackers to find a weak point, which helps protect customer data (Chourasia, 2023). It's also easier to add new features, like a new payment method, without breaking the rest of the system .

The disadvantage of this structure is that it can be slower. Since the separate parts need to send messages to each other instead of directly calling each other like in a monolithic system,

this adds extra delay This can be a problem for a platform that needs to handle a huge number of transactions quickly.

3.MONOLITHIC

Monolithic structure can help a cloud-based e-commerce platform to handle a thousands of transaction per second. Because in this structure, all the procedures like main program, service procedures, and utility procedures are linked in together in a one large binary running in the kernel mode. Since they are linked together, they share same address that's why service procedures can directly call each other instead of passing messages like other structure. Which makes the transactions like checkout, inventory updates, and etc. processed quickly. This is backed up by research comparing monolithic and microservices architectures, where the monolithic system actually had the lowest response time (0.76 seconds) and highest throughput (282.5 requests per second) under a load of 1,000 concurrent users (Marieska et al., 2025).

The disadvantage of using this is reliability. Since there aren't any clear boundaries between different procedures a bug in one service can affect other service which can lead to crashing of the system.

II. Recommend an Operating System Structure

For the client cloud-based e-commerce platform, a hybrid approach if not best ,is one of the best approaches in this operating system structure. Since the platform had so many work or things to do, no single pure structure can really satisfy it all at once. But a hybrid design can do this by combining the strengths of monolithic, microkernel, and modular structures applying each one where it fits the best.

Performance: The core functions of the system is processing customer orders and handling transactions, it should follow monolithic structure where components talk to each other directly. This keeps overhead to a minimum and lets the platform process thousands of transactions per second, which really matters especially when there's a promotion or sales.

Reliability and Security: Sensitive components, like payment processing and user authentication should follow a microkernel structure, running as isolated processes separate from the rest of the system. In this way, if a bug or a problem happens in of those components it won't take down the entire platform.

Modularity: Also the platform should support loadable modules similar to a modular structure, so new features like payment methods, or hardware/device support can be added or updated while the system is running. It doesn't need to shutdown or to restart the whole platform just to make update. It makes the client flexibility to scale and improve the system as the business keeps growing.

III. System Calls and Programs Mapping

1.Process Control

These system calls are used to create, manage and terminate processes on the server. On the e-commerce platform, whenever the customer checks out, the OS creates a new process to handle that specific transaction. This matters a lot because if a thousands of customers are checking out at the same time, each transaction needs to run as its own separate process. So if one fails it doesn't affect others.

2. File Management

These system calls handle creating, opening, reading, and writing files. On the platform, this is how customer order records and transaction log gets stored. For example, once an order is confirmed this is where the details were saved. It is also possible to check the customer's order history if you want to loop back for records or if there's a case of refunds.

3. Device Management

This manages access to hardware components like storage drivers and network interface cards. On the platform, the OS has to control which processes can access storage at the moment, so conflicts don't happen when multiple are reading or writing at the same time.

4. Information Maintenance

This is used to get or set system information like the time and the date. It is actually very helpful for timestamping every transaction, knowing exactly when an order or payment happened isn't just for keeping accurate records, but also to find out suspicious activity.

5. Communication

This lets different servers pass information to each other. On an e-commerce platform, the different parts of the system constantly need to coordinate with each other. For example, before an order gets confirmed, the payment server has to check with inventory server that the item's still in stock. Once the order is accepted, it also needs to notify shipping server so the package can start.

3 Essential System Programs

1. File Management Programs

These are the tools to create, copy, and organize files. Things like backup utilities or database management tools. They're essential for handling large volume of data on the e-commerce platform deals with, like customer records and product catalogs. In this way critical data isn't lost if something goes wrong.

2. Compilers/Interpreters (Programming-Language Support)

These are needed throughout the whole development lifecycle of the platform, from building new features to making updates. Let's say there's a case that the dev team is using Java or

Python to build the checkout system, they'll need a compiler to turn that code into something the system can actually run. This is also what makes debugging possible when something breaks in production.

3. Status Information / Monitoring Programs

These provides a real-time info about how the system's doing server load, error logs and etc. This makes proactive maintenance possible for example, if a monitoring tools show CPU usage near its limit the IT team can step up before the platform actually crashes. It also show's how activity gets logged, which is helpful for finding out what went wrong after the outage.

IV. Risk and Debugging Considerations

Possible Risks

One major of the major risk is when system crashes. If a bug shows up in the kernel let say somewhere in the file system, it can take down important services like checkout or payment processing, which directly means lost sales (GeeksforGeeks, 2023). This risk is even bigger in the monolithic part of the system, because everything shares the same address or space so one failure can affect others.

Another risk comes from bug in in drivers, especially the one that handles storage and network device. Since the platform is constantly reading and writing data , when processing orders, updating stocks a bug could occur here that could lead to a failed transactions or corrupted data, especially during high-traffic time like sales and promotions.

Debugging and Performance Tuning Strategies

To deal with kernel crashes, the system should rely on logging and crash dumps. When a crash happens, the error gets saved to a log file and to the system's memory. It helps the developers to analyze it and figure it out on what actually went wrong (GeeksforGeeks, 2023).

To catch performance issues before it gets bad , the system can use tools that can track activity in real time. These tools log important events along with their timing and details, which helps developers to find out what exactly the cause of slowing down (GeeksforGeeks, 2023). Tools like DTrace can even inspect the system while it's still running, without needing to shut anything down first (GeeksforGeeks, 2023).

Lastly, the hybrid structure itself helps reduce risk. By keeping sensitive parts like payment and login separate (microkernel-style), a bug in one part won't crash the whole system (Aalpha, n.d.). This is also supported by tools like Kubernetes, which can automatically restart failed services, reducing downtime (Bellundagi, 2022).

References:

Guntakandla, A. R. (2025). *Microservices and modular architecture: Revolutionizing e-commerce scalability*. Journal of Computer Science and Technology Studies. Al-Kindi Center for Research and Development.

https://www.researchgate.net/publication/391639545_Microservices_and_Modular_Architecture_Revolutionizing_E-Commerce_Scalability

IJEETR. (n.d.). *Design and implementation of scalable microservices architecture*. International Journal of Emerging Engineering Technologies and Research.

<https://ijeetr.com/index.php/ijeetr/article/download/490/454>

Chourasia, A. (2023). *Mastering software architecture: Part 4 The magic of microkernel architecture*. Medium. <https://ayoushchourasia.medium.com/mastering-software-architecture-part-4-the-magic-of-microkernel-architecture-e8f3ddc2571b>

Marieska, M. D., Yunanta, A., Aulia, H., Utami, A. S., & Rizqie, M. Q. (2025). Performance comparison of monolithic and microservices architectures in handling high-volume transactions. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 9(3), 594–600.

GeeksforGeeks. (2023). *Operating system debugging*.

<https://www.geeksforgeeks.org/operating-systems/operating-system-debugging/>

Aalpha. (n.d.). *Microkernel architecture pattern, principles, advantages & challenges*.

<https://www.aalpha.net/blog/microkernel-architecture/>

Bellundagi, M. (2022). Design and implementation of scalable microservices architecture for digital payment systems. *International Journal of Engineering & Extended Technologies Research*, 4(4), 5048–5054. <https://doi.org/10.15662/IJEETR.2022.0404007>